LAB 01

GIT COMMANDS REVIEW - PART1

Applied Data Science

Eman Hosam

AGENDA

- Introduction to Git and GitHub
- Git Installation and GitHub Registration
- Git Configuration
- Git Init and Clone
- Git Add and Commit
- Git Push and Pull
- Undo Changes in Git (Git reset)

GIT AND GITHUB

- **Version Control System:** a software that tracks changes to a set of files and resources.

- **Git** is a version control system.
- Other version control system: CVS, Subversion (SVN), Monotone, BitKeeper and Perforce.
- Git Advantages: free, open-source, fast, more compressed (space), and scalable.

- **GitHub** is a site for online storage of Git repositories.
- It is a platform for hosting and collaborating on Git repositories.

REPOSITORIES

- **Repository (repo):** a location storing a copy of all files of the different versions.

What to add to the repo?	What not to add to the repo?
- Source files: .cpp, .h, .java, etc. - Build files: Makefile, build.xml, etc. - Resource files: text files, images, etc.	- Files that are automatically generated: .obj, .exe, .class, etc.

- Two types of repos: local repo (on local machine) and remote repo (on GitHub).

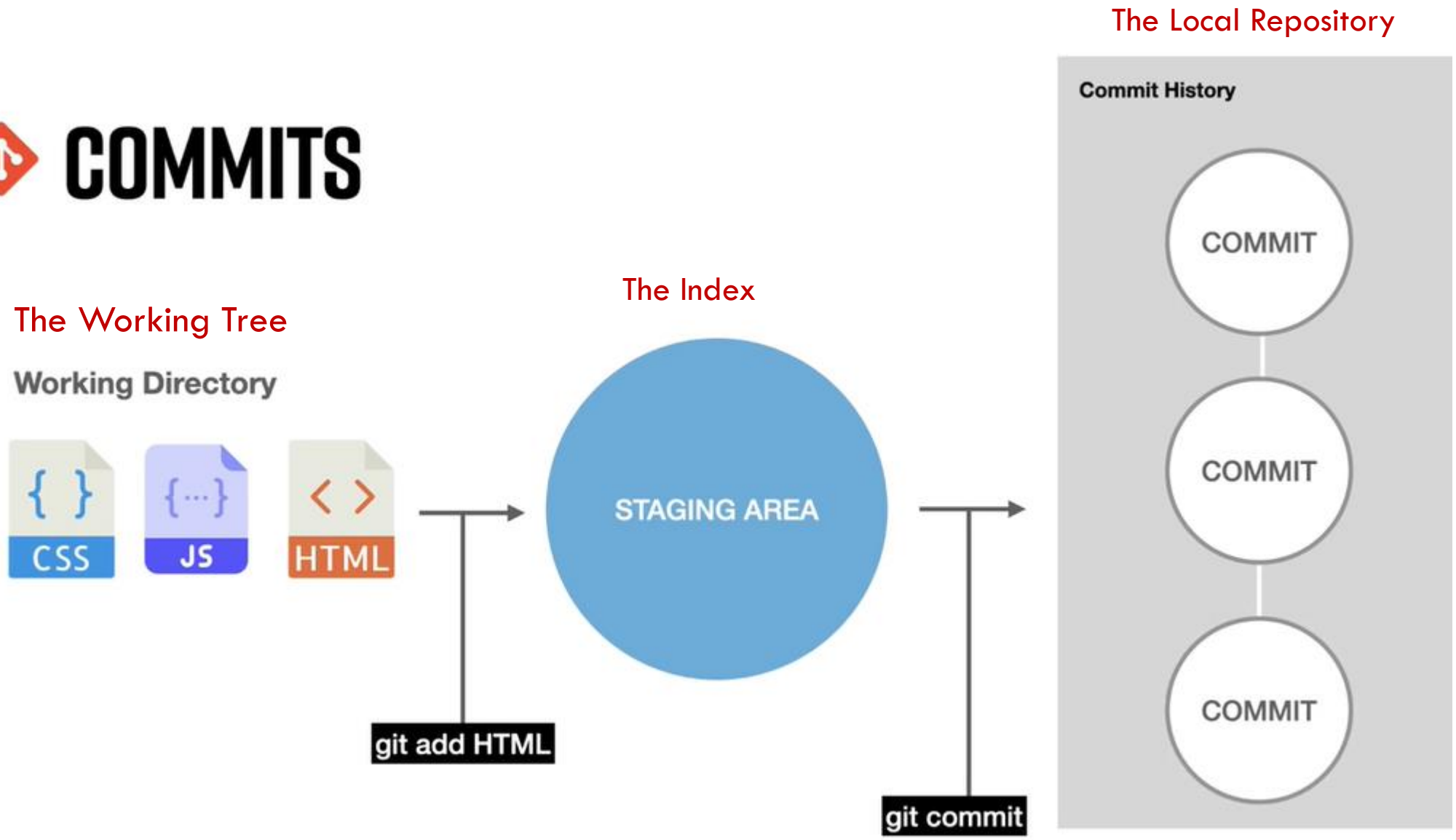
THE NEEDED INSTALLATION AND REGISTRATION

- Download and Install **Git:**
  http://git-scm.com/downloads

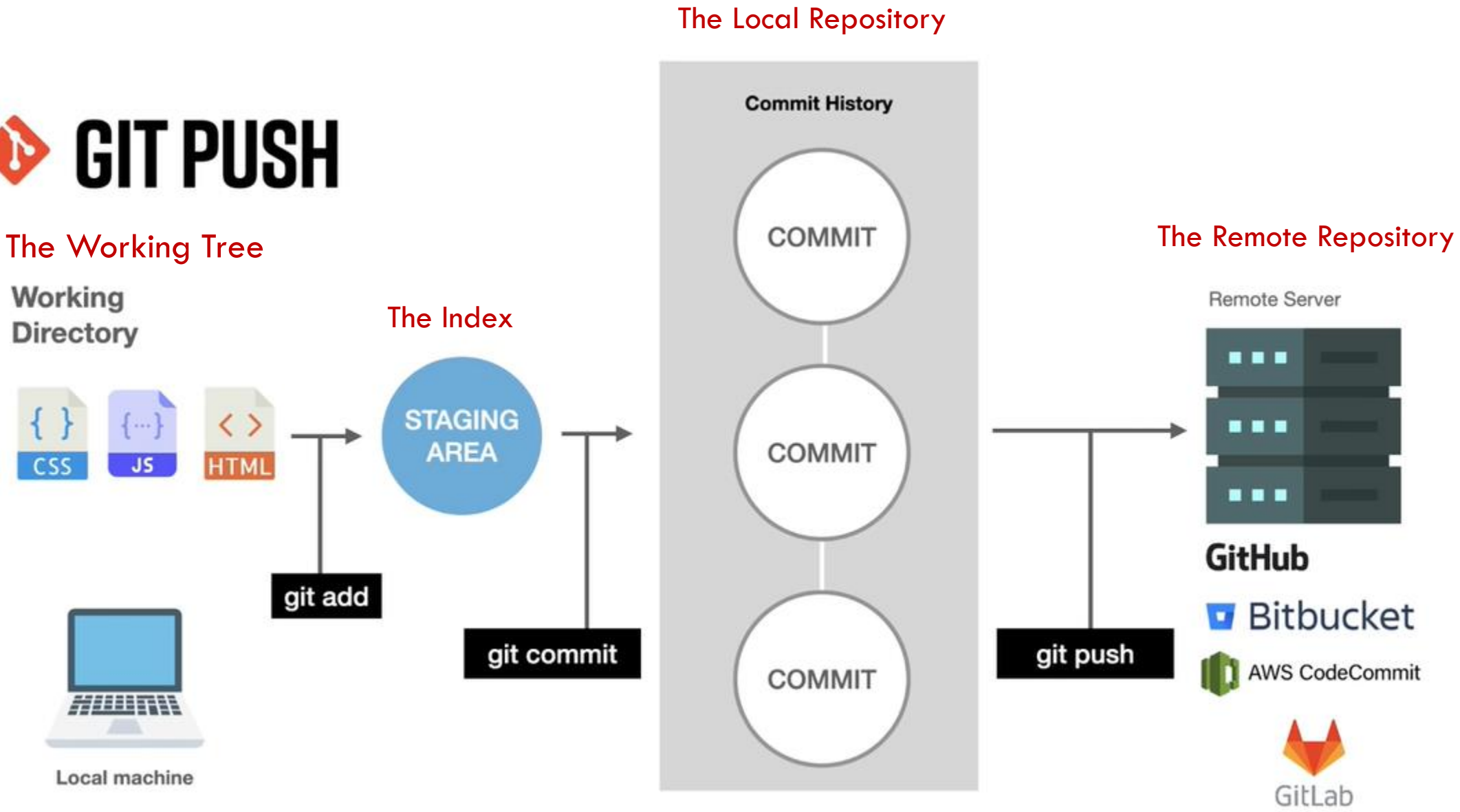
- Register freely on **GitHub:**
  https://github.com/

- The commands in the following slides are written inside the **Git Bash command line.**

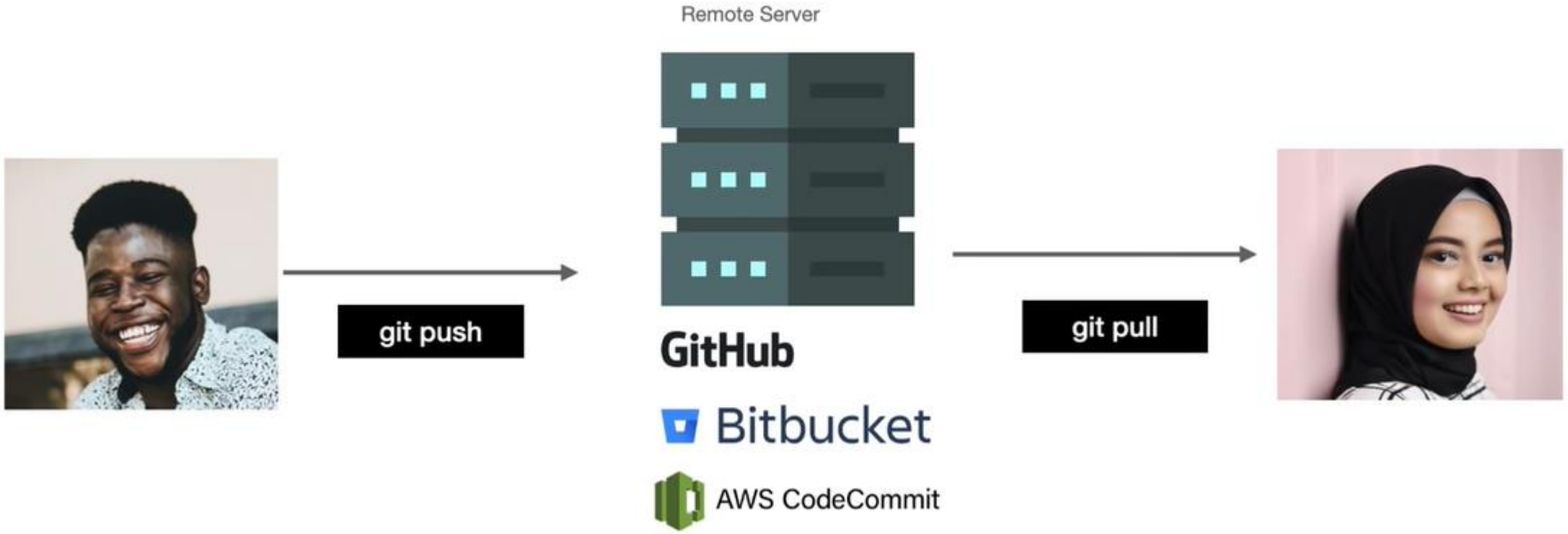
COMMITS


The Local Repository
Commit History
COMMIT
COMMIT
COMMIT
The Index
STAGING AREA
The Working Tree
Working Directory
CSS
JS
HTML
git add HTML
git commit

GIT PUSH


The Local Repository
The Working Tree
Working Directory
CSS
JS
HTML
Local machine
git add
The Index
STAGING AREA
git commit
Commit History
COMMIT
COMMIT
COMMIT
git push
The Remote Repository
Remote Server
GitHub
Bitbucket
AWS CodeCommit
GitLab

GIT PUSH & CLONE



GIT CONFIGURATION

GIT VERSION AND HELP

- $ git --version

  displays the installed version of git

```
Eman@EMAN-PC MINGW64 /
$ git --version
git version 2.32.0.windows.2
```

- $ git --help

  displays the git commands and how to use them

```
Eman@EMAN-PC MINGW64 /
$ git --help
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
           [--super-prefix=<path>] [--config-env=<name>=<envvar>]
           <command> [<args>]
```

GIT CONFIG

- $ git config --global user.name "emanhosam"

  sets the username of the git user

- $ git config --global user.email "eman.hosam@gmail.com"

  sets the email of the git user

```
Eman@EMAN-PC MINGW64 /
$ git config --global user.name "emanhosam"

Eman@EMAN-PC MINGW64 /
$ git config --global user.email "eman.hosam@gmail.com"
```

- Note: you need to do the above commands only once
- $ git config --global --list

```
Eman@EMAN-PC MINGW64 /
$ git config --global --list
user.name=emanhosam
user.email=eman.hosam@gmail.com
```

GIT INIT AND CLONE

CREATE A NEW REPOSITORY

- $ git init

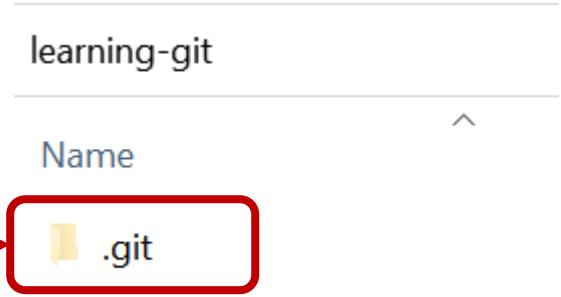
  makes the current directory a git repository (creates a new git repository)

```
Eman@EMAN-PC MINGW64 /
$ cd "C:\Users\Eman\Desktop\learning-git"
```

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git
$ git init
Initialized empty Git repository in C:/Users/Eman/Desktop/learning-git/.git/

Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ |
```

Or (main), both indicate that you are inside a repo.



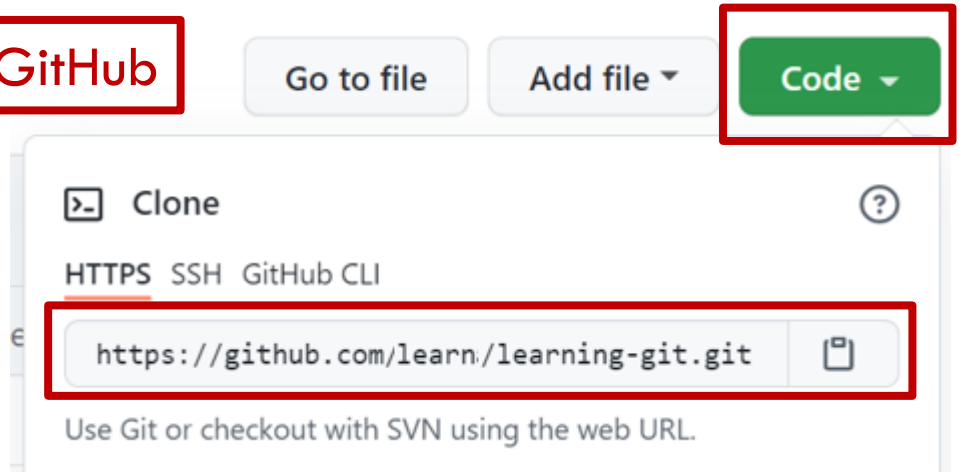
This is a hidden folder

DOWNLOAD A COPY OF AN EXISTING REPOSITORY

- $ git clone https://github.com/learn/learning-git.git

  Downloads a copy of a remote repository (from GitHub) into your local machine

GitHub

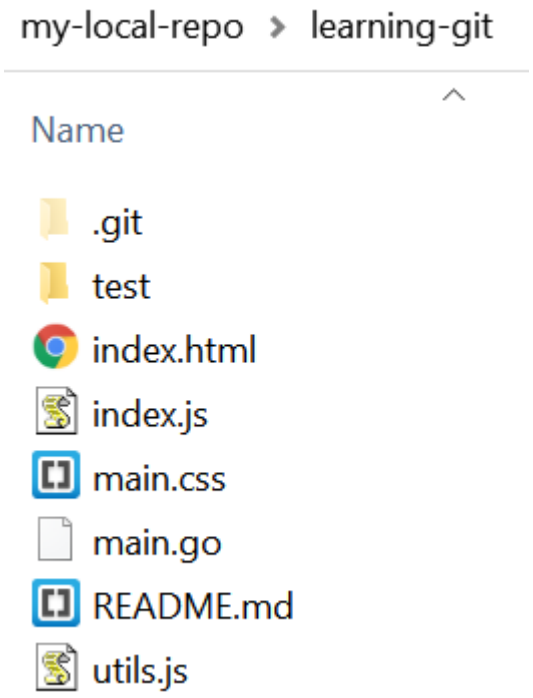


1

2

```
Eman@EMAN-PC MINGW64 ~/Desktop/my-local-repo
$ git clone https://github.com/learn/learning-git.git
Cloning into 'learning-git'...
remote: Enumerating objects: 36, done.
remote: Counting objects: 100% (36/36), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 36 (delta 12), reused 25 (delta 9), pack-reused 0
Receiving objects: 100% (36/36), 5.28 KiB | 772.00 KiB/s, done.
Resolving deltas: 100% (12/12), done.
```

3 Then, You will find the files downloaded.



GIT ADD AND COMMIT

DISPLAY GIT STATUS

- $ git status

  displays the status of the working directory and the staging area, etc. such as:

  - "untracked files": the new files that are not "added" yet to the staging area.
  - "changes not staged for commit": the modifications you made and not staged yet
    Note: the files under this category are already added before to the staging area but their content is updated without re-adding again (making a "git add" again).
  - "changes to be committed": the staging area files that need to be committed.

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        index.js
        main.css

nothing added to commit but untracked files present (use "git add" to track)
```

This means that the 3 files are not added/staged before.

ADD FILES TO THE STAGING AREA

- $ git add index.html main.css

  adds the specified files to the staging area (make them “tracked”/“staged”)

- $ git add .

  adds all the files recursively to the staging area

  - The period (.) is part of the command. It means all files/folders
  - Recursively means from this current directory and downwards

ADD FILES TO THE STAGING AREA

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git add index.html

Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.js
        main.css
```

After adding/staging "index.html", now it is ready for committing **"changes to be committed"**.

The other 2 files are still untracked.

ADD AND REMOVE FILES TO/FROM THE STAGING AREA

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git add .

Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
        new file:   index.js
        new file:   main.css
```

After adding/staging "all files, all now appear under "**changes to be committed**".

COMMIT TO THE LOCAL GIT REPOSITORY

- A commit is a safe point or a snapshot of the current project version.
- $ git commit -m "created a bootstrap project "

  commits the files of the staging area to your local git repository
  - provide a descriptive message that describes the changes made in this commit
- Note: after each update in the project, you should Add and Commit again.

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git commit -m "created a bootstrap project"
[master (root-commit) 3f20b8f] created a bootstrap project
 4 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
 create mode 100644 index.js
 create mode 100644 main.css
 create mode 100644 test/test.js

Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

This output indicates that all changes are committed to the local repo.

DISPLAY THE HISTORY OF COMMITS

- $ git log

  displays the list of all commits with information

  Note: the commits are displayed descendingly by time (the most recent first).

- $ git log --oneline

  like git log but displays each commit in one line with less details

- $ git show 3f20b8fa3f6eaf0a59098b685c6f30863162bddb

  displays the information of the passed commit id

```
Eman@EMAN-PC MINGW64 ~/Desktop/learning-git (master)
$ git log
commit d87ea73a2333e2f43cc9a57ab1bcf1dddc9cd8aa (HEAD -> master)
Author: emanhosam <eman.hosam@gmail.com>
Date:   Thu Sep 2 04:32:49 2021 +0200

    added console.log

commit 3f20b8fa3f6eaf0a59098b685c6f30863162bddb
Author: emanhosam <eman.hosam@gmail.com>
Date:   Thu Sep 2 03:52:34 2021 +0200

    created a bootstrap project
```

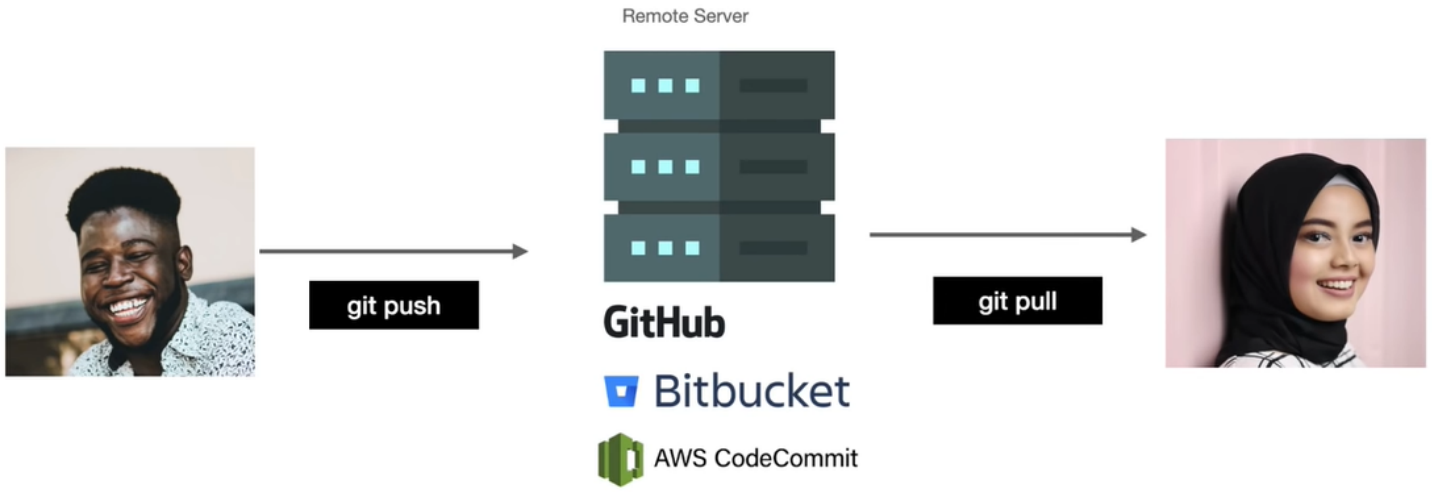
GIT PUSH AND PULL

- $ git push

  pushes the updates from the local repository to the remote repository (GitHub)

- $ git pull

  pulls the updates from the remote repository (GitHub) to the local repository



UNDO CHANGES IN GIT (GIT RESET)

HOW TO UNDO A COMMIT (OPTION 1: --SOFT)

- $ git reset --soft HEAD~2

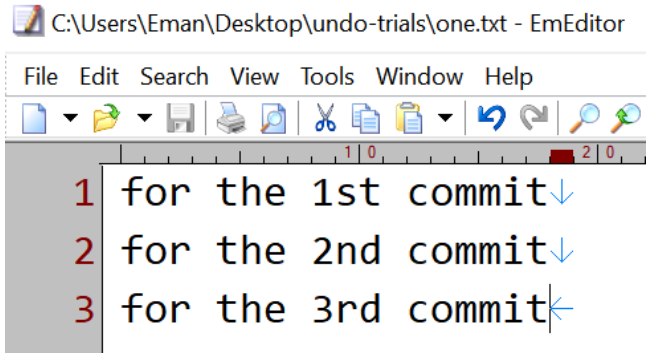
  removes the changes of the last 2 commits and point to the commit before
- Note: the changes of them are still staged.
- Note: 2 could be any number N.

```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git reset --soft HEAD~2
```

```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git log --oneline
2976eae (HEAD -> master) first commit

Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   one.txt
```

There were initially 3 commits and after resetting 2 commits using --soft, only the 1st remained but the changes are still staged.



HOW TO UNDO A COMMIT (OPTION 2: --MIXED)

- $ git reset --mixed HEAD~2

  or just $ git reset HEAD~2

  removes & unstages the changes of the last 2 commits and point to the commit before

- Note: the changes of them are unstaged but still kept in the working area.

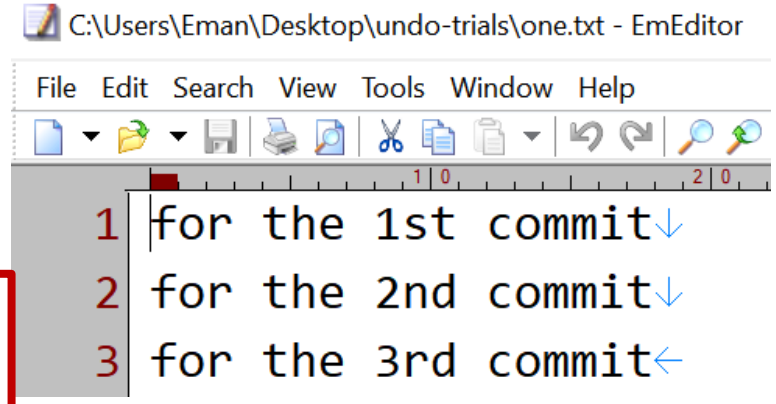
```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git reset --mixed HEAD~2
Unstaged changes after reset:
M       one.txt
```

```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git log --oneline
91a5ca2 (HEAD -> master) first commit

Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   one.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Note that the changes become not staged but still exist in the file of the working area



HOW TO UNDO A COMMIT (OPTION 3: --HARD)

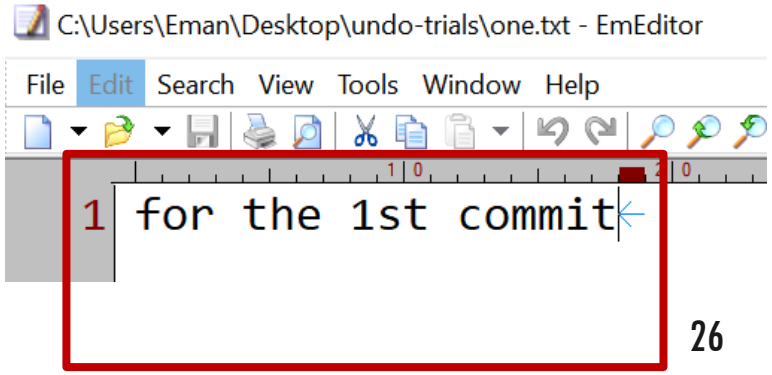
- $ git reset --hard HEAD~2

  removes the changes of the last 2 commits from (i) the repository, (ii) the staging area and (iii) the working area then point to the commit before them

- Note: the changes of them are unstaged and removed from the working area.

```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git reset --hard HEAD~2
HEAD is now at 91a5ca2 first commit
```

```
Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git log --oneline
91a5ca2 (HEAD -> master) first commit

Eman@EMAN-PC MINGW64 ~/Desktop/undo-trials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

The changes of the last 2 commits are removed from all places even the files of the working area.